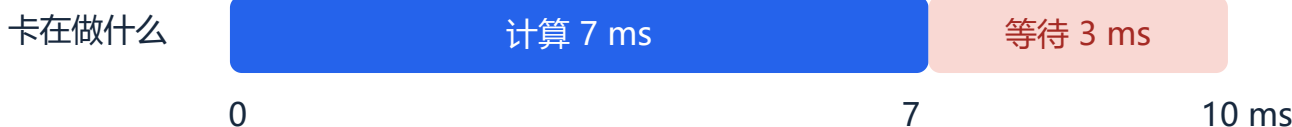


先看懂：NPU 到底在忙什么？

从一张卡开始，不需要先懂公式。

NPU 可以理解成负责计算的“工位”。它需要的数据还没送到时，即使有请求排队，也可能只能等。



这一段总共 10 ms；其中真正计算了 7 ms。

利用率 = 真正计算的时间 ÷ 观察的总时间

这个小例子：7 ÷ 10 = 70%。

本项目说的“平均 NPU 利用率”，统计的是**计算时间占比**。它不是磁盘带宽使用率，也不是硬件峰值算力使用率。

整本教程只围绕一件事：**下一步要计算时，数据到了没有？**

阅读路线

第 2-5 页：一条请求为什么等。

第 6-11 页：32 张卡为什么会一起堵，以及会不会恢复。

第 12-16 页：全程利用率、开环闭环、Once 和 TTFT。

第 17-18 页：公式速查与来源。

输入长，不一定算得久

本次只用两种真实请求：A 和 B。

一次输入中，一部分旧内容的 KV 可以复用，另一部分需要新计算。本实验中，可复用的 KV 要从 SSU 读取。SSU 就是提供数据的存储设备。

A：读得多，算得短

总输入 128K

其中只新增 256 token

175.66 MiB

每层读取量

6.02 ms

每层计算时间

B：读得少，算得久

总输入 32K

其中新增 4096 token

38.50 MiB

每层读取量

28.59 ms

每层计算时间

token 是文本的计量单位；这里 $K=1024$ 。miss 指需要新计算的部分：若用百分比表示，就是“新增 token 数 ÷ 总 token 数”。

不要只说“长流、短流”。A 的总输入更长，但计算反而更短。后面固定使用蓝色 A、绿色 B。

读取量、计算时间均取原始 data，没有缩放。若 KV 已在 NPU 本地显存，就不应再算一次 SSU 读取；本页描述的是这次实验设置。

为什么能一边计算，一边读取？

先把“模型层”和“调度层次”分开。

本次一个请求要依次算 **8 个模型层**，像完成 8 道顺序工序。不是一次读完、算完就结束。



当前层开始计算时，可以提前读取下一层的数据。这叫**预取**。只要下一层数据在当前计算结束前到齐，读取就被“藏”进了计算时间。

这张 NPU

算当前层：4 ms

等数据：4 ms

下一层读取

从发起到收齐：8 ms

0

4

8 ms

两件事同时开始；要两件事都结束，才能算下一层。

等待 = 读取总耗时 - 能重叠的计算时间

若结果小于 0，就按 0 算。图中：8 - 4 = 4 ms。

读取总耗时包括**排队和传输**；下方灰条不表示磁盘一直在为这条请求工作。

这是教学例子。起点是假设当前层数据已到齐。第一层没有前驱请求时，还需要先拿到首层数据。

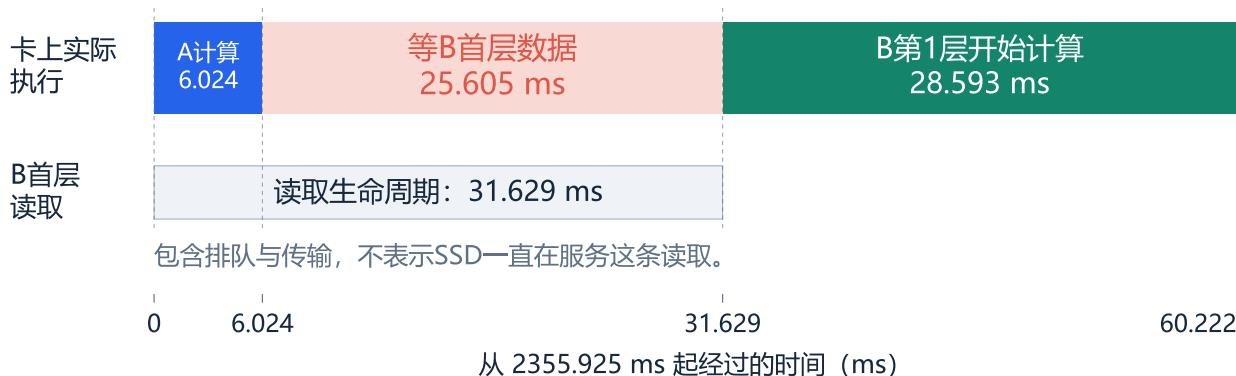
B 算得久，为什么首层还是会等？

这次实验的真实事件：前一个 A，把卡交给 B。

B 首层在 A 最后一层计算开始时就发起预取。但 A 只算 6.02 ms；B 自己的 28.59 ms 计算，此时还没开始。

4盘 Ordered · NPU 15 · A15000009 → B15000010

下一条B的首层读取，在前驱A的末层计算开始时提前发出



A末层=日志L7；B首层=日志L0。这是A→B交接的局部示例。

真实事件按同一个起点展示：NPU15，B 请求 15000010；前驱为 A 请求 15000009。

$$\text{B 首层等待} = 31.63 - 6.02 = 25.61 \text{ ms}$$

31.63 ms 是从发起读取到 NPU 收齐的总耗时，包含排队。

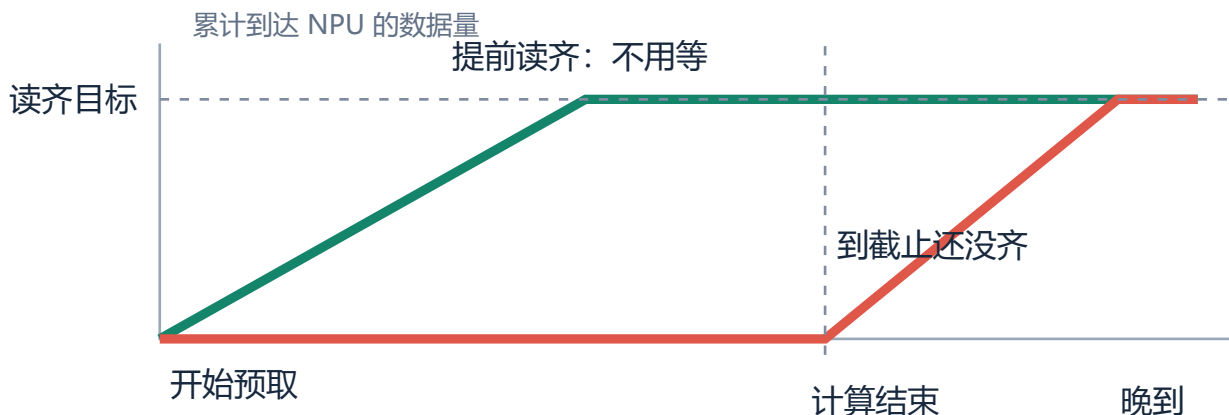
B 自己算得久，不能帮助它隐藏开始计算之前的首层等待。它只能借用前一个 A 最后一层的计算时间。

A 计算结束时，B 所需的 38.5 MiB 还一字节都没到 NPU。B 只能等到收齐后再开始计算。B 的后续层能借用自己的长计算时间，情况可能很好。

“需要多快” 和 “实际多快” 是两件事

先看截止时有没有收齐，再看带宽曲线。

假设要在 4 秒计算期间拿到 40 份材料，平均需要 10 份/秒。但并不要求**每一秒**都恰好送 10 份：前 2 秒送齐，也完全可以。



需求

想在多久内，拿到多少数据。

实际供给

SSU 真正读出多少；经过链路后，NPU 真正收到多少。

判断 IO stall 的方法：**计算结束时，下一步需要的数据还没收齐**。后来突然以很高带宽补齐，也不能追回已经等掉的时间。

本次 A 的内部层要在约 6.02 ms 内读完 175.66 MiB，参考需求约 **28.48 GiB/s**。B 内部层的参考需求约 **1.31 GiB/s**；A→B 首层则不能使用 B 自己的计算时间。

L1、L2、L3：只是三个不同决定

它们不是神经网络的第 1、2、3 层。

L1 · 分卡

这条请求交给哪张 NPU？

例如：把某条 A 交给 NPU0，把另一条 B 交给 NPU1。

↓ 每张卡有了自己的待办队列

L2 · 排请求

同一张卡先处理谁、后处理谁？

例如：NPU0 执行 $A \rightarrow B \rightarrow B$ ，或 $B \rightarrow A \rightarrow B$ 。

↓ 当前计算和预取产生读取请求

L3 · 排读取

已经提交的读取块，怎样获得盘的服务？

Baseline 在每块 SSU 的普通队列 Path0 按块排队，先到先服务，叫 FIFO。每块盘各有自己的队列。

L3 让数据早点到，可能让下一条请求早点开始；但这**不等于修改了 L2 的请求名单**。L3 也不能直接读取尚未提交的未来数据。

32 张卡，这次到底怎样喂请求？

同一批请求，每卡 40 个 A、80 个 B；只改变排列。

Ordered: 32 张卡都执行 $A \rightarrow B \rightarrow B$ ，重复 40 轮。下面是原始输入的前 12 条：

NPU 0	A	B	B	A	B	B	A	B	B	A	B	B	...
NPU 1	A	B	B	A	B	B	A	B	B	A	B	B	...
NPU 16	A	B	B	A	B	B	A	B	B	A	B	B	...
NPU 31	A	B	B	A	B	B	A	B	B	A	B	B	...

Random: 每张卡独立打乱自己的同一批请求。下面同样直接取原始输入：

NPU 0	B	B	B	B	A	A	B	A	B	A	B	A	...
NPU 1	B	B	B	A	B	A	A	B	B	A	B	A	...
NPU 16	B	B	A	B	A	B	B	A	B	B	A	B	...
NPU 31	B	B	B	A	B	A	A	B	B	A	B	B	...

两组都不是“某张卡只跑 A，另一张卡只跑 B”。**每张卡都混合两种请求。**

全部请求在 $t=0$ 入队；各卡按自己的进度推进，没有运行时同步屏障。Ordered 是刻意相关的压力输入；Random 使用 seed7。前期筛选人口较短，本教程结果统一使用上述长验证人口。

只改变顺序，结果就不一样

4 块 SSU，每盘 40 GiB/s；32 张 NPU，每卡链路 50 GiB/s。

Random · 随机

99.48%

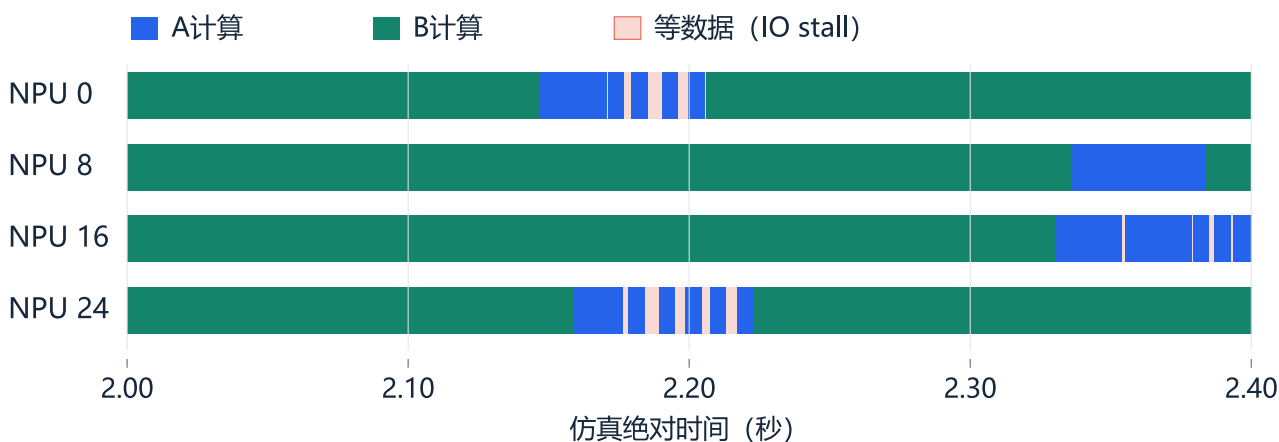
Ordered · ABB

70.23%

上面是全部 32 卡在 2~4 秒的平均利用率；均为 seed7。

4盘 · Baseline · Random (随机顺序)

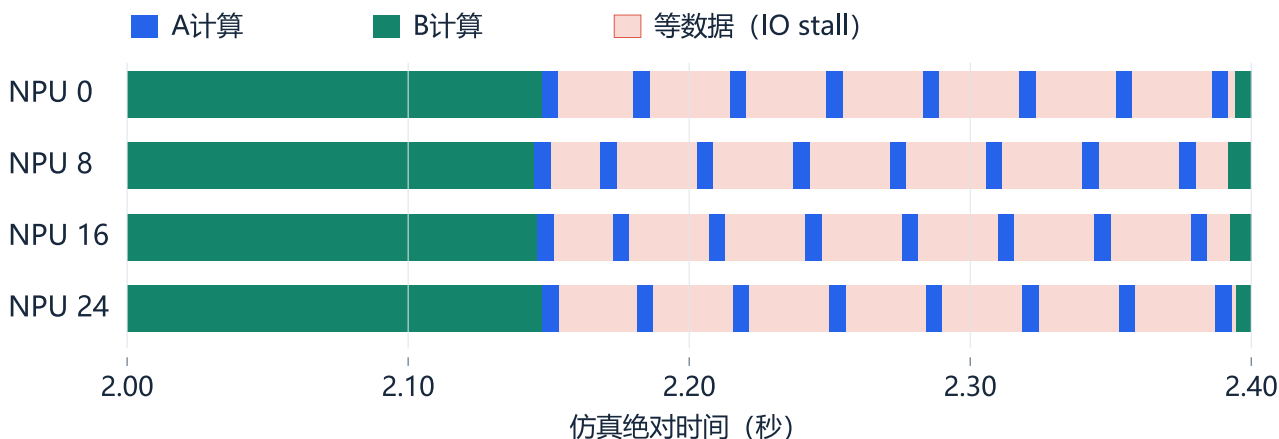
4卡节选，不等于整机利用率



Random: 只节选 4 张卡在 2.0~2.4 秒的真实计算与等待。

4盘 · Baseline · Ordered (A、B、B重复)

4卡节选，不等于整机利用率



Ordered: 使用相同卡号、相同绝对时间窗。蓝=A 计算，绿=B 计算，浅红=IO 等待。

图帮助看发生了什么；百分比来自完整 32 卡、完整统计窗，不是只对图里的四张卡求平均。

在 2~4 秒内，两组全部 32 卡都有任务，且每卡都实际计算过 A、B。当前 27 格全部使用 Baseline，尚未比较本次输入的 Once。

真实结果：4 盘长验证 seed7；统计窗 [2,4) 秒。Random 不是三种子平均。

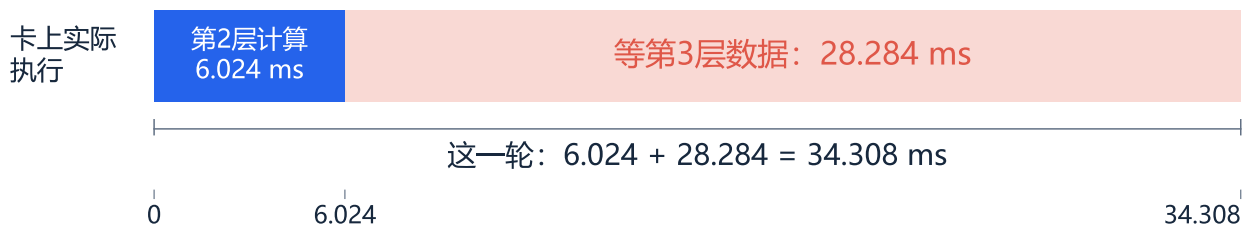
有一小段，利用率真的只有 17.56%

看清原因：这时 32 张卡都处于大读取的 A 阶段。

四盘合计 160 GiB/s，且一直忙。在这个稳定读取轮次里，每张卡平均得到 $160 \div 32 = 5$ GiB/s。注意：这是轮次平均，实际服务仍是一块一块发生。

4盘 Ordered · NPU 0 · A请求 9

真实绝对时间：2180.093–2214.401 ms



从本次读取发出起经过的时间 (ms)

第2层=日志L1，第3层=日志L2。这里只解释一个真实局部周期。

真实 NPU0 的完整交接：当前层计算结束后，继续等下一层数据。

每卡读 175.66 MiB，按周期平均 5 GiB/s，需要 34.31 ms。

这段只算 6.02 ms，其余约 28.28 ms 都在等。

$$\text{利用率} = 6.02 \div 34.31 \approx 17.56\%$$

完整精度计算为 17.558692%；用两位小数手算会略有差异。

此时各卡的层开始时间已经错开，仍然会等。“错开”不是“数据一定来得及”的保证。

该共同窗口的 32 卡实际利用率也为 17.56%。但这仅是 2.180093–2.214401 秒的局部，不是整场 U。全 A 的需求本就远超容量，不能把这段全部损失都算成 FIFO 独有缺点。

盘总体够快，也可能来不及

先用一个小例子理解 FIFO 的弱点。

想象一块盘每秒送 10 份材料。大读取需 40 份，但有 20 秒计算可等；小读取需 10 份，却只有 2 秒计算可等。

$$\text{两卡参考需求} = 40 \div 20 + 10 \div 2 = 7 \text{ 份/秒}$$

7 小于盘容量 10：通过平均速率的检查。

大读取先

大读取：40 份 / 4 秒

小读

小读取先

小读

大读取：40 份 / 4 秒

小卡 2 秒后就要数据

全读完都是 5 秒

大读取先做：占 4 秒，小读取再做 1 秒。小卡在第 2 秒就算完了，却要等到第 5 秒才能拿齐，等了 3 秒。

小读取先做：1 秒就送齐，满足它的 2 秒截止；大读取第 5 秒完成，也赶得上它的 20 秒截止。

FIFO 只认先后，不知道谁更急。相同工作量、相同总带宽，交付时机不同，等待就可能不同。

这是一次交接的教学构造：两份读取预先就绪，明确全部大块先或全部小块先，忽略链路。不能冒充本次 AB 的实测欠载反例；本次 27 格均有逐盘名义需求超限。

看久一点，会不会自己恢复？

会不会恢复要看数据，不能只看一张短截图。

下图从同一批长运行日志，连续取 2 秒窗口。窗口内全部 32 卡一直有任务；三条线均为 Ordered、seed7。



情况	2-4 秒 U	18-20 秒 U	2-20 秒 U
3 盘	71.01%	67.86%	64.22%
4 盘	70.23%	70.23%	71.34%
6 盘	86.59%	100.00%	97.24%

6 盘会恢复：各卡逐渐在不同时间进入 A，大读取不再那么集中。**4 盘在已测长窗仍较低：**本次没有同样恢复到满利用率。

不同卡、不同请求发生等待，会改变后续读取的发起时刻。这个反馈既可能让它们散开，也可能维持拥挤轮次。

本批在理想持续计算时平均需要约 124.91 GiB/s。3 盘总共只有 120，平均已经不足；4 盘 160、6 盘 240 虽通过平均检查，仍不能保证每盘每刻欠载。

同样的活，为什么全程利用率不同？

“工作量一样” 没有规定 “多久才能做完”。

两种顺序都完成同一批 **3840 个请求**，总读取都是 **2526.56 GiB**，总计算都是 **647.27 卡·秒**。

“卡·秒” 就是把所有卡的计算时间加起来：两张卡各算 1 秒，合计 2 卡·秒。

4 盘 Random

20.64 秒完成

4 盘 Ordered

28.20 秒完成

$$\text{全程 } U = \text{总计算卡} \cdot \text{秒} \div (32 \times \text{全部完成时间})$$

同一人口 / seed7	Random	Ordered
开始到全部完成	98.02%	71.72%
共同暖窗 2-4 秒	99.48%	70.23%
共同长窗 2-20 秒	98.96%	71.34%

等待拉长完成时间，即使总计算量没变，分母也会变大。**同样工作量不能推出同样利用率。**

全程包含启动等待和尾部空闲；暖窗只看选定时间。任务做完后的空闲不能假装成 IO stall。有限人口的守恒式也不能直接套进任意 2 秒窗口。

开环、闭环，究竟在说哪里？

先问“谁在决定下一次工作何时出现”。

看外部请求到达

按时来：开环输入

例如每 100 ms 来一条，不管上一条是否完成。完成得慢，门口的队伍会变长。

还是看外部请求到达

做完再来：闭环输入

例如始终保留 32 条，完成一条才补一条。做得慢，后续进入的请求也会减少。

本次还要单独看：内部层读取

这一层开始计算时，就发起下一层读取。

算完后，数据没齐才继续等。本次所有外部请求在 $t=0$ 已入队，但内部 IO 并没有全部同时发出；它会跟随计算推进。

开环或闭环**不改变利用率的计算公式**；它改变请求、IO 何时出现，以及等待会不会影响下一次工作。

“有限请求”是另一回事：名单会不会用完。本次是**有限待办队列，内部 IO 推进有反馈**。不要只贴一个“开环/闭环”标签就停止分析。

若只改到达时间标签，任务始终可用、顺序和预取条件也相同，U 可以不变；若必须等完成才产生下一请求，就可能失去跨请求预取。两种情况要分开。

Once per layer 想改变什么？

它主要改变 L3：已提交的读取怎样排队。

Baseline

每块盘的全部 IO 都进入自己的 Path0。

Path0

A A B A

盘按这条队列的块先后服务。

Once per layer

每个请求的每层、每块盘规划一次，将块分到允许使用的队列。

队列 1

A B

队列 2

A A

它依据采集到的压力快照和预计服务情况规划；规划本层时会更新本地预计排队量。项目这里使用 **5 ms 快照**。每条队列内部仍是 FIFO，由盘在队列之间分配服务机会。

多条队列不会把一块 40 GiB/s 的盘变成 80 GiB/s。它想改变的是**谁先得到数据**，从而减少错过计算截止的情况。

“每层规划一次”不表示整层只走一条队列。**同一层可以使用多条允许的队列**。本页讨论的是项目现有 **once** 策略。

本次 AB 的 27 格只测了 Baseline。我们还不知道 Once 对这批输入能改善多少；多队列图只是机制示意，不表示它必然按 A/B 各分一条队列。

利用率高，用户就等得短吗？

不一定：利用率看卡，TTFT 看请求。

TTFT 是用户从发出请求到收到第一个输出 token 的时间。本仿真没有测实际首 token 返回，而是用 prefill 完成作为代理；还必须说明从哪里开始计时。



同一条 B 请求	计时结果	SLO×1.5
只从接纳开始	231.51 ms	达标
从原始到达开始	2264.68 ms	不达标

本请求纯计算约为 8×28.59 ms，允许用时为纯计算的 1.5 倍，即约 **343.11 ms**。只计接纳后就达标，但把前面的队伍算进来就不达标。

本项目常用的高达标率是**接纳后处理时间代理**。不能据此说用户从到达到返回也很快。

而且 A 的允许时间只有约 **72.29 ms**。应分 A/B 看等待和达标率；总平均可能掩盖某类请求的问题。

这里用 6 盘 Ordered 的 NPU0 / B 请求 10 演示。暖窗按接纳时间选请求并追踪到完成，各策略选中的人口可能不同；公平比较还需匹配请求或外部到达轨迹。

读完实验，现在能下什么结论？

把事实、解释和下一步分开。

已测事实

相同每卡人口，排列会显著改变 Baseline。

4 盘的 2-20 秒 U: Random 98.96%, Ordered 71.34%。6 盘 Ordered 会自然恢复；不能把它的短窗低值当成长期表现。

有日志支持的解释

读取集中 + 计算预算短，会反复错过截止。

FIFO 参与决定块的服务先后。容量、预取边界和输入相位也参与其中，不能把所有损失都归给“只有一条 path”。

还缺的关键实验

同一输入，把 L3 换成 Once。

固定 4 盘现成输入，分别补 Random Once 和 Ordered Once；不重抽请求、不重排数据，再比 U、A/B 等待与 SLO。

固定输入	Baseline 已测 U (2-20 秒)	Once
Random	98.96%	待测
Ordered ABB	71.34%	待测

读任何低利用率图，都问三句：**谁在等？等哪一层的数据？为什么截止前没送齐？**

表中均为本次 4 盘、32 NPU、长验证 seed7。待测格没有填入旧画像的 Once 数字。

现在再读公式：先会这五条就够了

符号只是缩写；先用中文读懂，再代数字。

中文意思	简写	在本教程哪里见过
计算占了观察时间的多少	$U = \text{计算时间} / \text{总时间}$	第 1、8 页
算完时数据还没齐，要再等多久	$S = \max(0, R - C)$	第 3、4 页
计算和读取都完成，才进下一层	$T = \max(C, R)$	第 3、9 页
想在计算预算内读齐，要多快	需求 = V / C	第 5 页
同一批任务全部做完的平均 U	$U = W / (32 \times \text{完成时间})$	第 12 页

C：这次能用来隐藏读取的计算时间；跨请求时找前驱。

R：从发起读取到 NPU 收齐，包含排队与传输。

S：露在计算之外的等待；**T**：一次交接的总周期。

V：读取量，取 Volume（数据量）的首字母；原教程写 L ，含义相同。**W**：全部任务的计算时间相加，单位卡·秒。

只有在固定画像、供给规律明确的模型中，才可以继续推： $U \approx \text{实际带宽} \div \text{需求带宽}$ ，并封顶 100%。第 9 页的稳定轮次符合条件。

混合窗口不要直接除平均带宽：4 盘 Ordered、NPU15，那样算出 27.96%，实际为 70.23%。本次读取换算用 1 GiB=1024 MiB；带宽算式中的时间用秒。

回到原文时，从这些位置读

不需要一次记住 26 个编号。

原文公式	你现在应该怎样理解
1-3：输入与需求	hit 决定要复用的数据量；miss 影响计算。计算成本还受上下文等影响，直接用原始 data 更可靠。
4-8：读取、重叠、周期	排队不能漏；两件事并行取较慢的完成时刻。无预取的串行模型才把计算与读取相加。
9-12：平均与截止	数窗口内的计算；看截止前目标数据是否到齐。
13-16、26：容量与分配	容量检查不等于按时交付。优先低需求流只是一种有前提的模型，不能替代公平性和 SLO 目标。
17-21：有限工作量	全部读多少、全部算多久是固定的；完成时间不一定固定。
22-25：TTFT/SLO	明确计时起点、每条请求自己的阈值，以及统计了哪些请求。

哪些是教学图？ 第 1、3、5、10 页是便于理解的示意，第 6、13、14 页是机制示意。第 4、7-9、11-12 页使用本次真实输入或结果；第 15 页是实际请求事件。

可复查的项目文件

原文：docs/L1_L2_L3_NPU_图解教程.md

主结果：results/baseline_ab128_32_ratio12_20260912/report.md

详细公式核对：同目录 formula_review/README.md

数值核验：accounting_checks.json、handoff_checks.json

本 PDF 图源与构建脚本：docs/l1_l2_l3_beginner/

核对日期 2026-09-12。结果均来自既有仿真，本次只整理教程、重绘真实日志，没有启动新实验。教程数字保留适合阅读的小数位，原始精度在上述核验文件中。3 盘平均过载；本次 27 格均存在逐盘名义需求超限。